Python interview commands

Copy the plumbing; spend your attention on the transformation and debugging.

1. CLI + friendly errors

```
import argparse, csv, json
from pathlib import Path

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--input-dir", required=True,
        help="Folder of JSON files")
    parser.add_argument(
        "--output-csv", required=True,
        help="Output CSV file")
    args = parser.parse_args()
    try:
        convert(args.input_dir,
                args.output_csv)
    except (ValueError, OSError) as exc:
        parser.exit(1, f"Error: {exc}\n")

if __name__ == "__main__":
    main()
```

convert() is your transformation function. Missing required flags are handled by argparse; parser.exit writes the supplied error to stderr.

2. Sorted files + JSON

```
input_path = Path(input_dir)
files = sorted(
    p for p in input_path.glob("*.json")
    if p.is_file())
rows = []
for path in files:
    with path.open(encoding="utf-8") as f:
        data = json.load(f)
    rows.extend(extract_rows(data))
```

extract_rows() is the schema-specific work. Validate inputs before opening the CSV. Raise a useful ValueError for bad data or no records.

3. Run / inspect help

```
python convert.py --input-dir raw \
    --output-csv output/train.csv
python convert.py --help
```

Full working copyable example: templates/convert_template.py. It documents its example schema; replace extract_rows() for a new contract.

4. Create parents + write CSV

```
output_path = Path(output_csv)
output_path.parent.mkdir(
    parents=True, exist_ok=True)
with output_path.open(
    "w", encoding="utf-8",
    newline="") as csv_file:
    writer = csv.writer(csv_file)
    writer.writerow(["text", "label"])
    writer.writerows(rows)
```

Let csv handle commas, quotes and embedded newlines. Count records with csv.DictReader, not physical file lines.

5. Last FINAL / key membership

```
latest = {}
for review in reviews:
    if review["status"] == "final":
        latest[review["id"]] = review["label"]
if identifier in latest:  # label 0 is valid
    label = latest[identifier]
```

Reset per file if IDs are local to each export. Last in list order and greatest timestamp are different rules.

6. Strings / sets

```
changed = text[:i] + "X" + text[i+1:]
changed = text.replace(old, new, 1)
seen = set()       # membership
lookup = {}        # ID -> value
```

Strings are immutable. Preserve a separate list when output order matters. Use is None or is True when the contract requires identity.

7. breakpoint() - short version

Insert breakpoint() before the suspect line. p EXPR prints; n is next; s steps into; c continues; q quits. l shows source; w shows stack; u/d changes frames. Inspect type(x), x.shape, x.dtype and x.device.

8. Minimal training loop

```
import torch.nn.functional as F
optimizer.zero_grad(set_to_none=True)
logits = model(**inputs)  # tensor wrapper
loss = F.cross_entropy(logits, labels)
loss.backward()
optimizer.step()
```

A raw Hugging Face model usually returns an object: use outputs.logits. Our ML wrappers already return a tensor. backward computes gradients; step updates managed parameters.